

Buffer overflows and memory protection

Tom Chothia
Lecture 2

Buffer Overflows

- The instruction pointer controls which code executes,
- The instruction pointer is stored on the stack,
- I can write to the stack ... 😊

Buffers 32-bit

1. Function called with "Hello World" ...
`function (user input);`
2. Arg, EIP & EBP written to stack ...
3. Function runs `function (char *str) {`
4. Buffer allocated `char buffer[16];`
5. String copied `strcpy(str, buffer);`
`}`

	Hello World	Old EBP	Old EIP	Hello World	Stack
--	-------------	---------	---------	-------------	-------

Buffer Overflow 32-bit

If user input is more than 16 bytes

1. Runs as before
2. But the string flows over the end of the buffer
3. EIP corrupted, segmentation fault

```
...  
function (user input);  
...  
  
function (char *str) {  
    char buffer[16];  
    strcpy(buffer, str);  
}
```



Once more, with malice

1. Runs as before
2. Attacker sends a very long message, ending with the address of some code that gives him a shell.
 - The attacker's code could also be part of the message
3. The attacker's value is copied over the old EIP
4. When the function returns the attacker's code is run

	Hello World	XXXXXX	7797F9	HelloWorldXX XXXX7797F9	Stack
--	-------------	--------	--------	----------------------------	-------

What To Inject

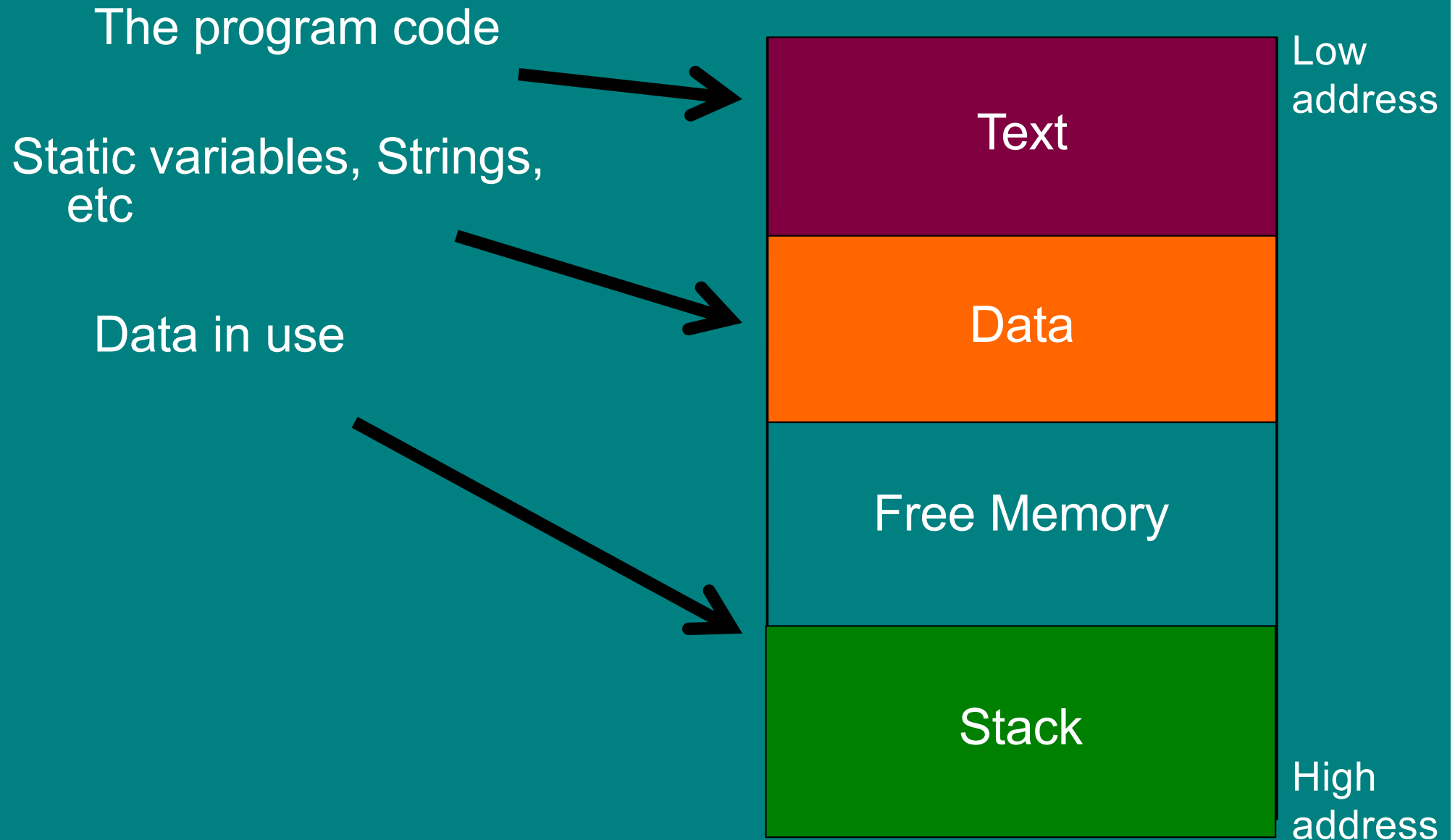
0:	31	c9				xor	ecx,ecx
2:	f7	e1				mul	ecx
4:	b0	0b				mov	al,0xb
6:	51					push	ecx
7:	68	2f	2f	73	68	push	0x68732f2f
c:	68	2f	62	69	6e	push	0x6e69622f
11:	89	e3				mov	ebx,esp
13:	cd	80				int	0x80

\x31\xC9\xF7\xE1\xB0\x0B\x51\x68\x2F\x2F\x73\x68\x68\x2F\x62\x69\x6E\x89\xE3\xCD\x80

Buffer Overflows in ARM

- The return address is in the `lr` register!
- When a function calls another, `lr` is put on the stack.
 - Target this.
- The overflowed function will return correctly
- Caller function will put the overwritten value into `lr`.
- When the caller function returns, the attack's code will run.

Defense: The NX-bit

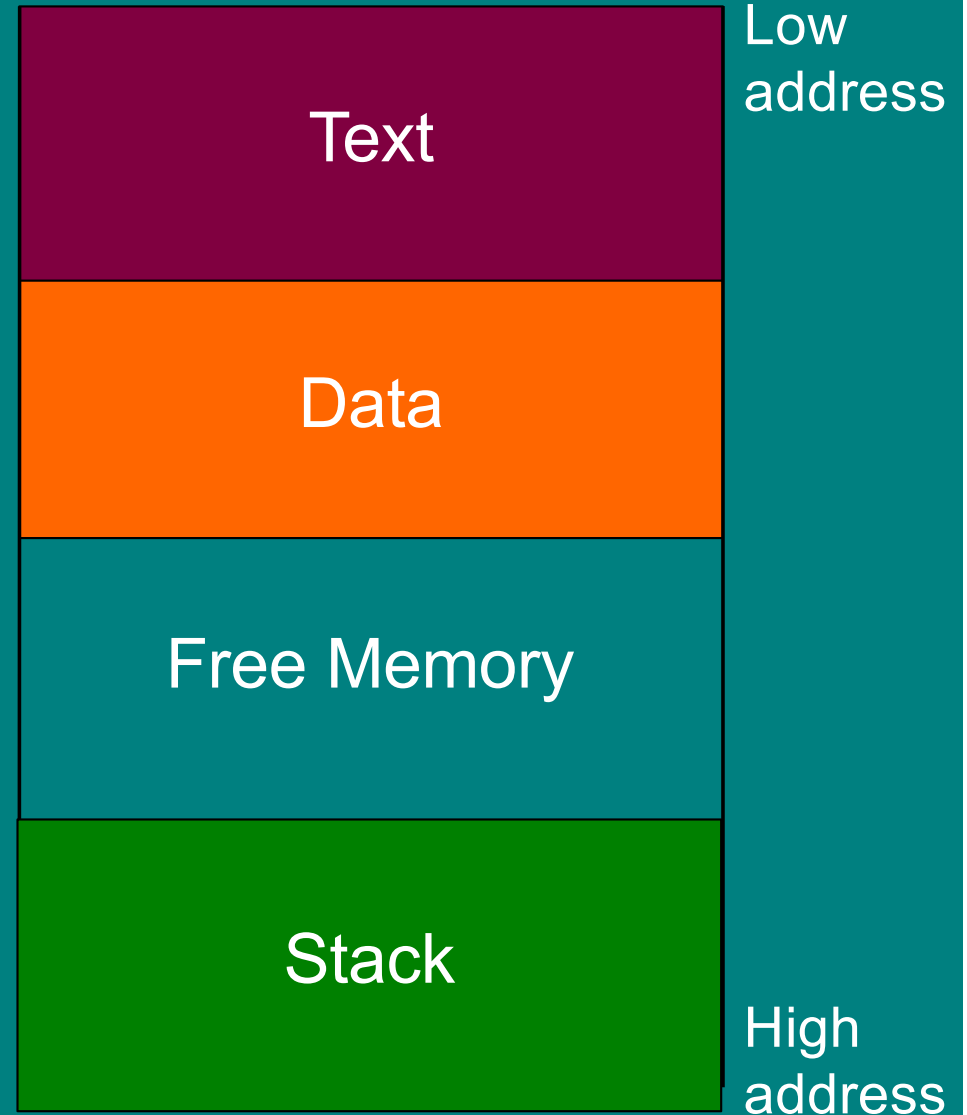


Defense: The NX-bit

The program code

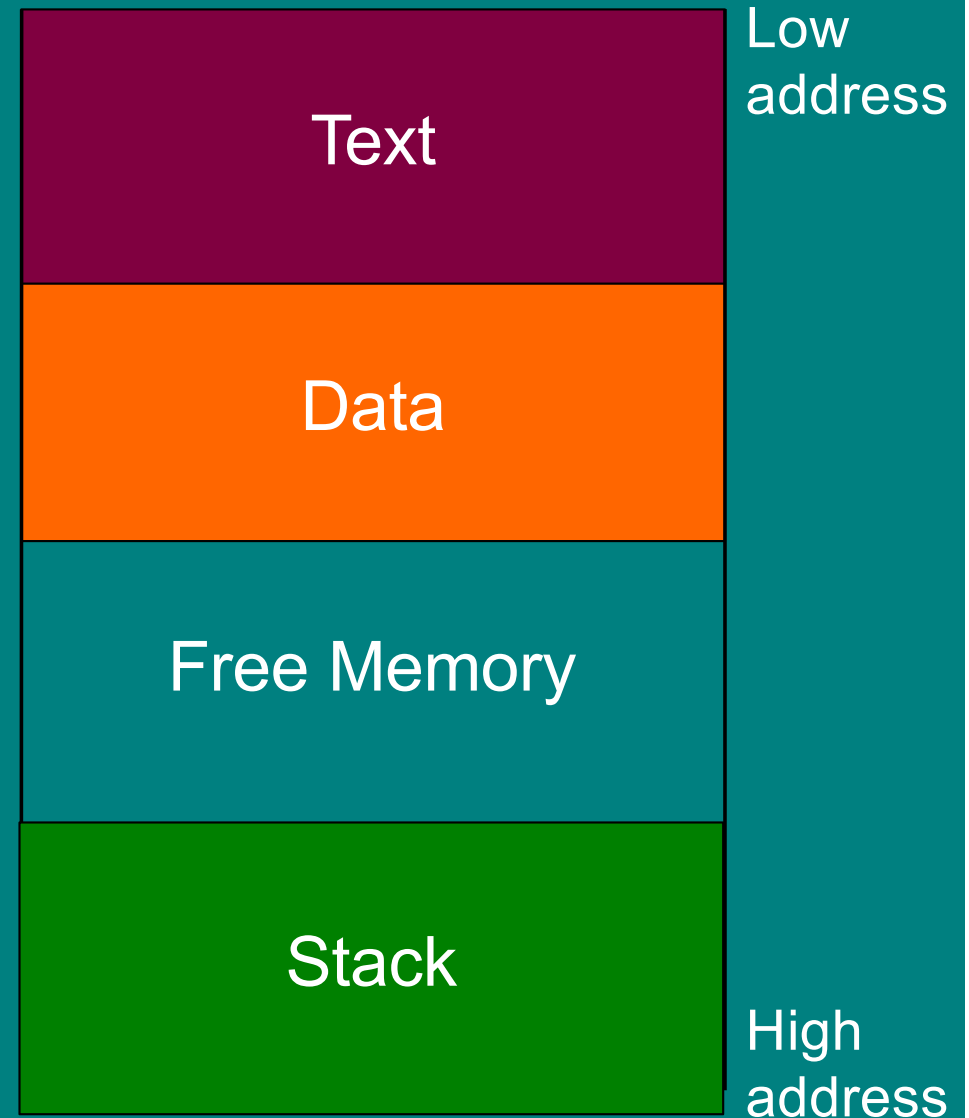


I injected code here



Defense: The NX-bit

- Code should be in the text area of the memory. Not on the stack.
- The NX-bit provides a hardware distinction between the text and stack.
- When enabled, the program will crash if the EIP ever points to the stack.



Address space layout randomization.

- ASLR adds a random offset to the stack and where the code is loaded each time the program runs.
- Jumps in the program are altered to point to the right line.
- The idea is that it's now hard for an attacker to guess the address of where they inject code or the address of functions.
- On by default in all OS
 - Off in Linux: `sudo echo 0 > /proc/sys/kernel/randomize_va_space`
 - On in Linux: `sudo echo 1 > /proc/sys/kernel/randomize_va_space`

Stack Canaries

- At the start of a function a random value from the heap is written to the base of the stack.
- When the function finishes the value on the stack is checked against the value on the heap.
- A overwrite attempt will overwrite the value on the stack, leading to a mismatch.

Stack Canaries

- When the program is loaded into memory, the kernel writes a random number to the stack canary address.
- The same random number is used by every function.
- Seen in windows: the stack canary is xor-ed with the stack base pointer (RBP). To make it different for every function.

Pascal

YOU WANT THE TRUTH?

**YOU CAN'T HANDLE THE
TRUTH!!!**

DIYLOL.COM

What I want you to know:

- How you can inject code and run it against a program with no defense.
- How W^X protection works.
- How Address Space Layout Randomization (ASLR) works.
- How stack canaries work.